BCA182 Embedded Systems Programming — Laboratory Activity No. 1

Real-Time Multisensor Room Monitoring System (Academic Laboratory Report)

Student: Binyusoph Sansarona | Institution: MSU-IIT (College of Computer Studies - Dept. of Computer Applications)
Instructor: Asst. Prof. Paul Rodolf P. Castor, M.Sc. | Target MCU: STM32F103C8T6 (Blue Pill) | Framework: STM32Cube HAL + FreeRTOS

GitHub Repository: https://github.com/BinSan21/FreeRTOS_Room_Monitoring_Multisensor

1. Problem and Requirements

1.1 Problem Statement & Wokwi Adaptation: Modern indoor environments require deterministic, concurrent monitoring of thermal conditions, relative humidity, ambient lighting, and human occupancy while minimizing unnecessary display power consumption during vacant periods. A single-threaded super-loop architecture struggles to balance microsecond-level sensor protocols (such as the DHT22 single-wire waveform) against high-frequency quadrature rotary encoder polling (5 ms) and slow I2C OLED frame buffer updates without introducing input lag or timing drift. To solve this, the system is designed and implemented from scratch on the STM32F103C8T6 (Blue Pill) ARM Cortex-M3 microcontroller using the STM32Cube HAL framework and native FreeRTOS kernel APIs, simulated within Wokwi.

1.2 Functional Requirements & System Parameters: The system satisfies ten core functional requirements (FR-01 through FR-10) with configured temperature alarm limits of LOW_TEMPERATURE_LIMIT = 18.0 C and HIGH_TEMPERATURE_LIMIT = 30.0 C, and an automatic inactivity timeout of 15,000 ms (15 seconds).

2. System Architecture and Design

2.1 Hardware & Modular Software Architecture (Part XIII): The hardware architecture integrates six external peripherals around the 72 MHz STM32F103C8T6 microcontroller. The software architecture is partitioned into eight modular C source and header pairs across include/ and src/ (main.c, rtos_objects.c, sensors.c, display.c, input.c, alarm.c, motion.c, and system_state.c). Pure decision logic (evaluateAlarmState, nextDisplayMode, previousDisplayMode, evaluateSystemState) is strictly isolated from STM32 HAL peripheral calls using #ifndef UNIT_TEST guards so deterministic host-based unit tests can execute natively.

Peripheral	Wokwi Component	STM32 Pin(s)	STM32 HAL Interface	System Role
STM32 Blue Pill	board-stm32-bluepill	-	72 MHz ARM Cortex-M3	Main MCU executing FreeRTOS scheduler
DHT22 Sensor	wokwi-dht22	PA1	GPIO Open-Drain + TIM2 (1 us)	Periodic temperature (C) & humidity (%) sensing
LDR Sensor	wokwi-photoresistor-sensor	PA0 (AO)	ADC1_IN0 (12-bit, 0-4095)	Relative ambient light measurement (0-100%)
SSD1306 OLED	board-ssd1306	PB6 (SCL), PB7 (SDA)	I2C1 (400 kHz, Addr 0x3C)	Single-metric user display (Temp/Hum/Light/Motion)
KY-040 Encoder	wokwi-ky-040	PB12 (CLK), PB13 (DT), PB14 (SW)	GPIO Input with Pull-Up	Clockwise / Counter-clockwise page selection
Piezo Buzzer	wokwi-buzzer	PA8	GPIO Output Push-Pull	Audible alert when Temp < 18.0 C or > 30.0 C
PIR Sensor	wokwi-pir-motion-sensor	PA4	GPIO Input with Pull-Down	Occupancy detection & INACTIVE wake-up trigger
Serial Monitor	USART1	PA9 (TX), PA10 (RX)	UART 115200 8N1	Mutex-protected diagnostic telemetry logging

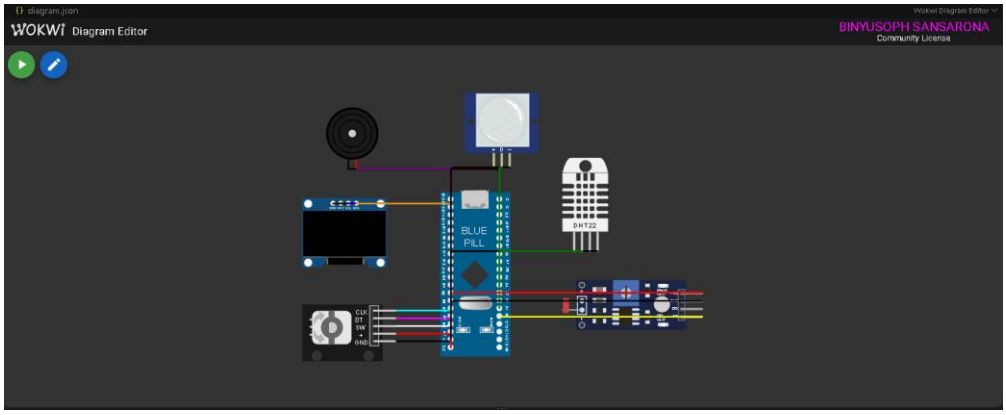


Figure 1: Wokwi Hardware Circuit Schematic for the STM32F103C8T6 Multisensor Room Monitor.

2.2 System State Machine Design (Part IX): The system implements a two-state occupancy state machine (SYSTEM_STATE_ACTIVE and SYSTEM_STATE_INACTIVE) in evaluateSystemState(). In ACTIVE state, the SSD1306 OLED is enabled (0xAF), rotary encoder navigation is polled every 5 ms, and sensor/alarm monitoring operate normally. When no motion is detected (motionDetected == false) for >= 15,000 ms, the system transitions to INACTIVE, clearing EVENT_ACTIVE, commanding the SSD1306 OLED into hardware sleep (0xAE), and suspending encoder page updates while keeping background PIR and thermal safety active. Any subsequent PIR motion immediately restores SYSTEM_STATE_ACTIVE.

3. FreeRTOS Architecture

3.1 Required FreeRTOS Task Table & Priority Justification (Parts XII & XX): The application is structured as six concurrent FreeRTOS tasks scheduled preemptively (configUSE_PREEMPTION = 1, 1000 Hz tick rate). Below is the required FreeRTOS Task Table (Part XX, Item 59) detailing each task, its responsibility, trigger/period, priority, IPC mechanism, and typical blocked condition:

Task	Responsibility	Trigger / Period	Priority	IPC Mechanism	Typical Blocked Condition
vSensorTask	Read DHT22 & LDR sensors; publish SensorData	Periodic: 2000 ms (vTaskDelayUntil)	2 (Medium)	xSensorDisplayQueue, xSensorAlarmQueue	Blocked on vTaskDelayUntil() timer expiration
vDisplayTask	Sole owner of I2C1 OLED; render pages or blank screen	Event / Queue update (50 ms timeout)	1 (Lowest)	xSensorDisplayQueue, xDisplayModeQueue, xSystemEventGroup	Blocked waiting for queue data or 50 ms frame delay
vInputTask	Decode KY-040 quadrature rotary encoder transitions	Short periodic: 5 ms (vTaskDelay)	3 (Highest)	xDisplayModeQueue, xSystemEventGroup	Blocked on 5 ms vTaskDelay() between pin polls
vMotionTask	Monitor PA4 PIR sensor & signal occupancy	Periodic: 100 ms (vTaskDelay)	3 (Highest)	xSystemEventGroup (EVENT_MOTION)	Blocked on 100 ms vTaskDelay() between PIR reads
vAlarmTask	Evaluate temperature thresholds & drive PA8 buzzer	Event: Sensor queue update (portMAX_DELAY)	2 (Medium)	xSensorAlarmQueue, xSystemEventGroup (EVENT_ALARM)	Blocked indefinitely on xQueueReceive() waiting for data
vStateTask	Evaluate 15 s inactivity timeout & ACTIVE/INACTIVE state	Periodic: 200 ms (vTaskDelay)	2 (Medium)	xSystemEventGroup (EVENT_ACTIVE, EVENT_MOTION)	Blocked on 200 ms vTaskDelay()

Technical Justification of Task Priorities: Priority Justification (Part XII, Item 39): Priorities are assigned strictly according to scheduling urgency and acceptable latency. vInputTask and vMotionTask are assigned Priority 3 (Highest) because mechanical rotary encoder quadrature pulses last only a few milliseconds and PIR wake-up events require immediate responsiveness; each executes in microseconds before blocking, so they never starve lower priorities. vSensorTask, vAlarmTask, and vStateTask share Priority 2 (Medium) to guarantee deterministic 2-second sampling and prompt thermal alarm actuation. vDisplayTask is assigned Priority 1 (Lowest) because human visual perception easily tolerates tens of milliseconds of latency during I2C1 frame transfers; if vDisplayTask had a higher priority than vInputTask, multi-millisecond I2C transfers would block rotary encoder sampling and cause missed detents.

3.2 Task States (Running, Ready, Blocked, Suspended, Deleted): During execution, each task transitions cleanly among FreeRTOS task states: Running (actively executing on the Cortex-M3 CPU), Ready (unblocked and waiting in the ready list while a higher- or equal-priority task holds the CPU), Blocked (consuming zero CPU cycles while waiting on vTaskDelayUntil, vTaskDelay, xQueueReceive, or xSemaphoreTake), Suspended (not used directly; logical inactivity is coordinated via EVENT_ACTIVE), and Deleted (tasks run in infinite loops and are never deleted).

3.3 Why vTaskDelayUntil() Prevents Periodic Drift (Part IV, Item 23): vTaskDelayUntil(&xLastWakeTime, pdMS_TO_TICKS(2000)) is used in vSensorTask instead of vTaskDelay() because vTaskDelay(2000) delays relative to the moment vTaskDelay() is called. Since DHT22 bit-banging, ADC conversion, and queue publishing take variable execution time (t_exec), using vTaskDelay(2000) results in an actual period of (2000 ms + t_exec + preemption_delay), causing cumulative timing drift over time. In contrast, vTaskDelayUntil() increments the previous wake tick reference (xLastWakeTime += 2000) and wakes the task at exact absolute 2000 ms multiples regardless of how long readSensors() took to execute, eliminating cumulative sampling drift.

3.4 Mutex Shared-Resource Protection & Failure Mode Analysis (Part XI, Item 37): Shared Resource: Hardware peripheral USART1 (accessed via _write / printf / Serial_Printf). Competing Tasks: All six tasks (vSensorTask, vDisplayTask, vInputTask, vMotionTask, vAlarmTask, and vStateTask) log state transitions and telemetry over UART. Failure Mode Prevented: Because UART transmission at 115200 baud takes ~87 microseconds per byte, a higher-priority task (e.g., vMotionTask at Priority 3) can preempt a lower-priority task (e.g., vSensorTask at Priority 2) mid-sentence while vSensorTask is halfway through shifting characters into USART1->DR. Without xSerialMutex, characters from both tasks interleave on the serial terminal (e.g., "[Sensor[Motion] PIR: YES Task] Temp: 25.4 C"), corrupting telemetry logs. Acquiring xSerialMutex via xSemaphoreTake(xSerialMutex, portMAX_DELAY) before formatting and transmitting guarantees atomic, serialized lines.

3.5 Event Group Signaling (Part X, Item 35): xSystemEventGroup coordinates three bits: EVENT_ACTIVE (Bit 0, set/cleared by vStateTask, read by vDisplayTask and vInputTask), EVENT_MOTION (Bit 1, set/cleared by vMotionTask, read by vStateTask and vDisplayTask), and EVENT_ALARM (Bit 2, set/cleared by vAlarmTask, read by vDisplayTask).

4. Implementation

Key Engineering & Implementation Decisions: 1. Dedicated OLED Ownership (display.c): vDisplayTask is the exclusive owner of I2C1 and the 1024-byte SSD1306 frame buffer. Rather than letting multiple tasks write directly to I2C1, producers push SensorData and DisplayMode items into queues. This architectural choice prevents I2C bus collisions without needing an I2C mutex.
2. Microsecond DHT22 Driver (sensors.c): Uses TIM2 configured with a prescaler of (72 - 1) for a 1 MHz (1 us) counter. During the 40-bit data burst, taskENTER_CRITICAL() briefly masks interrupts for ~4 ms so a FreeRTOS tick interrupt cannot stretch pulse-width measurements, while the preceding 18 ms start signal uses cooperative vTaskDelay(20) so other tasks run freely.
3. Quadrature State-Table Decoder (input.c): Instead of fragile edge-Counting that bounces on mechanical encoders, vInputTask shifts the 2-bit (CLK, DT) state into a 4-bit lookup index to reliably detect full clockwise and counter-clockwise detents.

4. Dual Sensor Queues (rtos_objects.c): Publishing SensorData to both xSensorDisplayQueue and xSensorAlarmQueue ensures that vAlarmTask wakes immediately upon every temperature reading without competing with vDisplayTask for the same queue item.

5. Verification and Testing

5.1 Automated Unit Testing (Part XIV — 14/14 Passed): Fourteen (14) host-based unit tests (exceeding the minimum requirement of 13) were implemented using the Unity framework under test/ and executed via pio test -e native. All 14 tests passed cleanly:

Suite	Test Function	Input Stimulus	Expected Output	Status
test_alarm	test_alarm_below_lower_threshold	17.9 C (< 18.0 C)	ALARM_STATE_LOW_TEMPERATURE	PASSED
test_alarm	test_alarm_at_exact_lower_threshold	18.0 C (== 18.0 C)	ALARM_STATE_NORMAL	PASSED
test_alarm	test_alarm_within_normal_range	25.4 C (normal)	ALARM_STATE_NORMAL	PASSED
test_alarm	test_alarm_at_exact_upper_threshold	30.0 C (== 30.0 C)	ALARM_STATE_NORMAL	PASSED
test_alarm	test_alarm_above_upper_threshold	30.1 C (> 30.0 C)	ALARM_STATE_HIGH_TEMPERATURE	PASSED
test_navigation	test_nav_clockwise_forward_transitions	TEMP -> HUM -> LIGHT -> MOTION	Sequential forward modes	PASSED
test_navigation	test_nav_clockwise_wraparound	nextDisplayMode(MOTION)	DISPLAY_MODE_TEMPERATURE	PASSED
test_navigation	test_nav_counterclockwise_reverse_transitions	MOTION -> LIGHT -> HUM -> TEMP	Sequential reverse modes	PASSED
test_navigation	test_nav_counterclockwise_wraparound	previousDisplayMode(TEMP)	DISPLAY_MODE_MOTION	PASSED
test_state	test_state_active_no_timeout_remains_active	(ACTIVE, false, 14999 ms)	SYSTEM_STATE_ACTIVE	PASSED
test_state	test_state_active_at_timeout_becomes_inactive	(ACTIVE, false, 15000 ms)	SYSTEM_STATE_INACTIVE	PASSED
test_state	test_state_active_with_motion_even_at_timeout_stays_active	(ACTIVE, true, 20000 ms)	SYSTEM_STATE_ACTIVE	PASSED
test_state	test_state_inactive_no_motion_remains_inactive	(INACTIVE, false, 30000 ms)	SYSTEM_STATE_INACTIVE	PASSED
test_state	test_state_inactive_with_motion_restores_active	(INACTIVE, true, 30000 ms)	SYSTEM_STATE_ACTIVE	PASSED

5.2 Wokwi Functional Verification Record (Part XVI — FT-01 to FT-10): All ten functional tests (FT-01 through FT-10) were executed in the Wokwi simulator and verified against expected behavior:

Test ID	Input / Stimulus	Expected Result	Actual Observed Behavior	Result
FT-01	Change DHT22 temperature slider (25.4 C)	Displayed temperature updates	OLED Temperature page & UART log updated to 25.4 C within 2 s	PASS
FT-02	Navigate to Humidity & adjust DHT22 humidity (61.2%)	Displayed humidity updates	OLED Humidity page updated to 61.2 % and matched UART log	PASS
FT-03	Navigate to Light & move LDR slider	Light value changes (0-100%)	OLED Light Level page updated dynamically to 76 %	PASS
FT-04	Rotate KY-040 encoder clockwise (CW)	Next page is selected with wraparound	Cycled Temp -> Hum -> Light -> Motion -> Temp cleanly	PASS
FT-05	Rotate KY-040 encoder	Previous page is selected	Cycled Temp -> Motion ->	PASS

	counterclockwise (CCW)	with wraparound	Light -> Hum -> Temp cleanly	
FT-06	Set DHT22 temperature above 30.0 C (32.5 C) / below 18.0 C	Alarm activates (Buzzer ON)	PA8 buzzer activated, EVENT_ALARM set, "[AlarmTask] HIGH/LOW" logged	PASS
FT-07	Return DHT22 temperature to normal (25.4 C)	Alarm stops (Buzzer OFF)	PA8 buzzer turned OFF, EVENT_ALARM cleared, NORMAL state restored	PASS
FT-08	Trigger PIR motion sensor during operation	System is ACTIVE & motion shown	EVENT_MOTION set, OLED Motion page showed DETECTED immediately	PASS
FT-09	Leave PIR untriggered for 15 seconds (15000 ms)	System becomes INACTIVE & blanks OLED	At 15000 ms without motion, StateTask logged INACTIVE & OLED slept	PASS
FT-10	Trigger PIR while system is INACTIVE	System returns to ACTIVE immediately	PIR pulse immediately woke OLED display and restored ACTIVE state	PASS

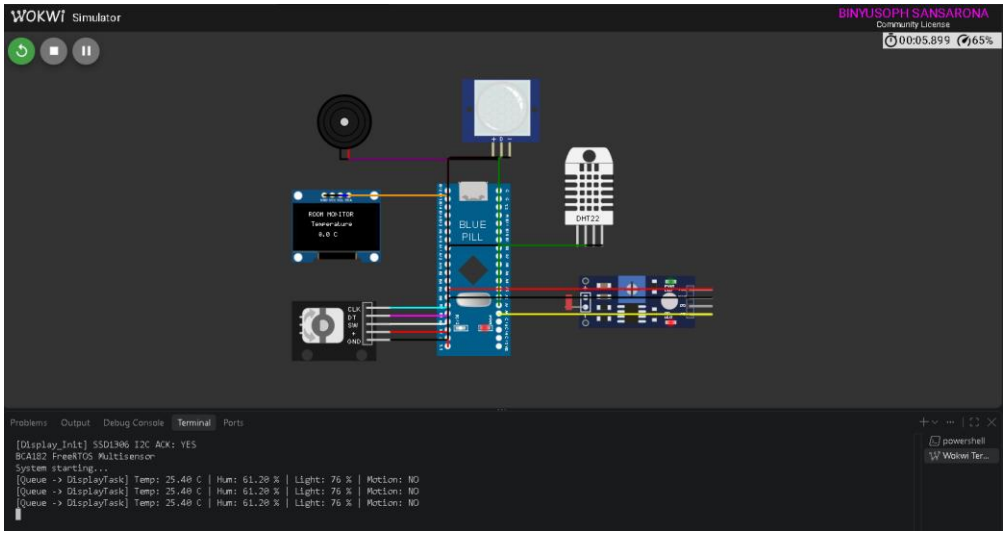


Figure 2: Finished-System Wokwi Simulation Screenshot Showing Live OLED Telemetry and Mutex-Protected UART Output.

5.3 Deliberate FreeRTOS Fault Experiments (Part XVII — Items 49–51): Three deliberate fault-injection experiments were conducted in Wokwi to observe real-time scheduling and synchronization failure modes (and restored afterward):

Experiment	**Injected Fault**	**Observed System Behavior**	**Technical Explanation & Root Cause**
Fault Experiment 1: Remove Blocking (Item 49)	Commented out vTaskDelay(pdMS_TO_TICKS(100)) inside vMotionTask (Priority 3).	System froze after boot message; vSensorTask (Prio 2), vAlarmTask (Prio 2), vStateTask (Prio 2), and vDisplayTask (Prio 1) never executed; OLED remained blank.	Because FreeRTOS uses fixed-priority preemptive scheduling, an unblocked infinite loop at Priority 3 consumes 100% of CPU time and permanently starves all Priority 2 and Priority 1 tasks. Restored vTaskDelay(100).
Fault Experiment 2: Change Priority (Item 50)	Elevated vDisplayTask from Priority 1 to Priority 4 (above vInputTask/vMotionTask) and removed its 50 ms block so it redrew I2C1 continuously.	Rotary encoder detents were frequently missed or delayed by several seconds, and PIR wake-up latency degraded noticeably.	Continuous multi-millisecond I2C1 frame transfers at Priority 4 preempted the 5 ms quadrature sampling in vInputTask (Priority 3), causing missed Gray-code transitions. Restored vDisplayTask to Priority 1.
Fault Experiment 3: Remove	Bypassed	Serial Monitor output exhibited	When a higher-priority task

Mutex (Item 51)	xSemaphoreTake/Give(xSerialMutex) inside Serial_Printf() while multiple tasks printed at 100-200 ms intervals.	corrupted, interleaved lines where strings from preempting tasks merged mid-line.	preempted a lower-priority task mid-UART transmission, both wrote concurrently to USART1->DR, creating a classic shared-peripheral race condition. Restored xSerialMutex.

6. Static Code Analysis

6.1 Static Analysis Inspection & Resolution (Part XV — Item 46): Static code analysis was executed using Cppcheck via pio check -e bluepill_f103c8. The initial scan reported 0 High and 0 Medium defects, and flagged 24 low-severity style warnings ([unusedFunction]). Inspection revealed one genuine obsolete function (Display_RenderTemperature in src/display.c:273) left over from Part VI before multi-page rendering was introduced, while the remaining 23 warnings were false positives caused by single-translation-unit scanning of cross-module APIs and FreeRTOS kernel hooks (_write, vApplication*Hook). Display_RenderTemperature was removed from display.h and display.c, and platformio.ini was configured to scope checks to project source files. Post-resolution pio check passed with 0 defects (0 High, 0 Medium, 0 Low).

Finding	File / Line	Severity & Cause	Corrective Action / Resolution
The function Display_RenderTemperature is never used [unusedFunction]	src/display.c:273	Low (Style): Obsolete single-page helper from Part VI superseded by Display_RenderScreen() in Part VII.	Deleted obsolete declaration from include/display.h and definition from src/display.c.
The function ... is never used [unusedFunction] (18 Cross-Module APIs)	src/*.c (alarm, display, input, motion, rtos_objects, sensors, system_state)	Low (Style False Positive): Cppcheck single-file translation unit scan lacks linker visibility across main.c and test/*.	Verified all 18 public module functions are actively invoked in main.c or unit tests; configured check_src_filters and --suppress=unusedFunction in platformio.ini.
The function ... is never used [unusedFunction] (5 Kernel/Syscall Hooks)	src/main.c:15, 255, 258, 265, 270	Low (Style False Positive): Invoked externally at link/runtime by Newlib printf (_write) and FreeRTOS kernel (vApplication*Hook).	Retained required RTOS safety hooks and syscall handlers; re-ran pio check -> 0 defects found.

7. Engineering Discussion & Requirements Traceability

7.1 Design Trade-Offs, Limitations & Debugging Insights: 1. Polling vs. EXTI Interrupts: Polling the rotary encoder at 5 ms and PIR sensor at 100 ms simplified debouncing and avoided ISR-flooding risks, at the cost of minor periodic CPU wake-ups.
2. Critical Section Duration in DHT22: Masking interrupts during the 40-bit DHT22 read (~4 ms) prevents RTOS tick preemption from corrupting 26-70 us bit pulses, trading a brief bounded scheduling jitter every 2 seconds for 100% reliable sensor checksums without needing DMA.
3. Simulated vs. Physical Hardware Differences: On physical STM32F103C8T6 silicon, mechanical KY-040 contacts exhibit 1-5 ms contact bounce requiring RC filtering or state-table rejection, ADC1 inputs require sample-time adjustment for high-impedance LDR voltage dividers, and DHT22 lines require a physical 4.7k-10k ohm pull-up resistor.

7.2 Requirements Traceability Matrix (Part XXI — Item 60): Every functional requirement (FR-01 to FR-10) is mapped below to its implementing module/task and verification evidence:

Requirement ID	Requirement Description	Implementation (Module / Task)	Verification Evidence
FR-01	Periodic temperature measurement	vSensorTask / DHT22_Read() (src/sensors.c)	FT-01 (Wokwi OLED & UART verification)
FR-02	Periodic humidity measurement	vSensorTask / DHT22_Read() (src/sensors.c)	FT-02 (Wokwi OLED & UART verification)

FR-03	Ambient-light level measurement (0-100%)	vSensorTask / LDR_ReadPercent() (src/sensors.c)	FT-03 (Wokwi LDR slider verification)
FR-04	Simulated motion detection via PIR sensor	vMotionTask / Motion_ReadPIR() (src/motion.c)	FT-08 (Wokwi PIR trigger & EVENT_MOTION)
FR-05	OLED display showing one selected metric at a time	vDisplayTask / Display_RenderScreen() (src/display.c)	FT-01, FT-02, FT-03, FT-08
FR-06	Rotary encoder CW/CCW navigation across 4 pages	vInputTask (src/input.c) & next/previousDisplayMode()	Unit navigation tests (4/4) + FT-04, FT-05
FR-07	Buzzer alarm when temperature outside 18.0 C - 30.0 C	vAlarmTask & evaluateAlarmState() (src/alarm.c)	Unit alarm tests (5/5) + FT-06, FT-07
FR-08	Support ACTIVE and INACTIVE system states	vStateTask & evaluateSystemState() (src/system_state.c)	Unit state tests (5/5) + FT-08, FT-09, FT-10
FR-09	Automatic transition to INACTIVE after 15 s no motion	vStateTask & evaluateSystemState() (src/system_state.c)	Unit state timeout test + FT-09
FR-10	Automatic reactivation to ACTIVE upon PIR motion	vMotionTask + vStateTask (EVENT_MOTION -> EVENT_ACTIVE)	Unit state wake-up test + FT-10

8. Conclusion & Incremental Git History

8.1 Lessons Learned & Future Improvements: This laboratory demonstrated how decomposing a multisensor embedded system into prioritized FreeRTOS tasks linked by queues, event groups, and mutexes eliminates timing drift, race conditions, and UI lag. Separating pure decision logic from STM32 HAL drivers enabled rapid host-side verification via 14 Unity unit tests and zero-defect Cppcheck static analysis. Future enhancements include migrating rotary encoder and PIR inputs to EXTI interrupts (xTaskNotifyFromISR) and enabling FreeRTOS tickless idle mode (__WFI) during INACTIVE periods.